npj | computational materials

Article

Published in partnership with the Shanghai Institute of Ceramics of the Chinese Academy of Sciences

https://doi.org/10.1038/s41524-025-01932-8

Graph embedding tensor: unifying topological description, symmetry detection, and structure generation for two-dimensional carbon allotropes

Check for updates

Lilac Macmillan[1], Eduardo Costa Girão[2] & Vincent Meunier[1]

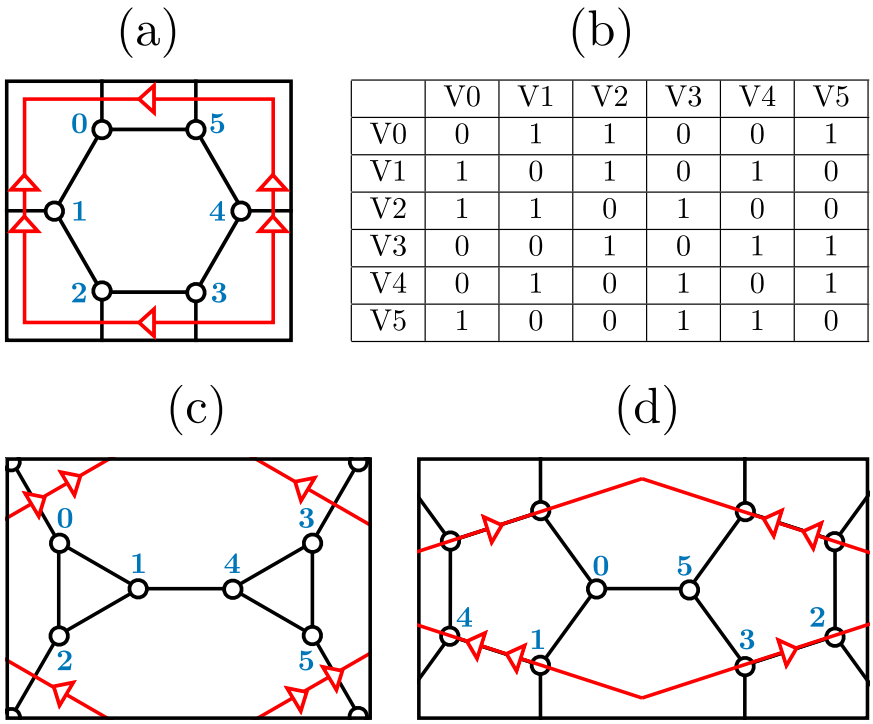
We introduce the *embedding tensor* Φ, a rank-3, purely integer tensor that elevates faces and edges to the same footing as vertices to provide a unique, coordinate-free representation of any three-connected two-dimensional carbon lattice. Defined by the incidence of vertices, edges, and polygonal faces, Φ obeys simple summation rules derived from Euler characteristics. Casting Φ into a *flag* graph enables exact, tolerance-free identification of wallpaper symmetries. Building on this algebraic framework, we develop an iterative add-dimer search that generates all structures with N_F faces from all crystals with $N_F - 1$ faces, while automatically discarding duplicates via tensor isomorphism and spotting non-primitive cells through symmetry checks. Exploiting symmetry keeps the combinatorial growth of dimer insertions tractable even for $N_F > 5$, transforming an otherwise exponential search into a practically feasible approach for high-throughput exploration. Once candidate topologies are enumerated, approximate real-space coordinates and lattice vectors can be reconstructed analytically from Φ and sparse crossing matrices, providing initial geometries for electronic or vibrational calculations. The method delivers an end-to-end pipeline from exhaustive, symmetry-aware enumeration to metadata tagging and coordinate generation, while requiring only integer arithmetic.

Graphene stands as the ultimate atomically thin two-dimensional material, consisting of a single atomic layer of carbon atoms arranged in a hexagonal honeycomb lattice[1]. As the archetypal 2D material, graphene's cone-shaped electronic band structure, with its linear dispersion relation near the Dirac points[2], has become emblematic of two-dimensional physics and appears in fundamental textbooks on condensed matter[3]. The unique electronic properties arising from this two-dimensional confinement have driven extensive research into band gap engineering and electronic structure manipulation within the planar carbon lattice, which has generated countless scientific investigations into the electronic behavior of this prototypical 2D system.

Apart from strategies that preserve the underlying graphitic lattice (such as nanotube/nanoribbon forms, defects, and doping), reorganizing sp^2 carbon in two dimensions in the form of alternative lattices has been a research topic of interest for a large community. The first contributions in this field date back to the mid-1990s[4] and early 2000s[5], with proposals based on the insertion of several reconstruction defects in graphene, while imposing the constraint of sp^2 hybridization defined by the threefold coordination of each atom. The basic idea is to introduce an increasing number of defects until we reach a point where the lattice cannot be easily recognized as a modified graphene. Different structures (also referred to as Haeckelites) are built based on distinct strategies to distribute such reconstructions[6], resulting in a rich family of systems and physical behaviors[7].

Several other 2D sp^2 nanocarbons have been proposed without mentioning extended modifications of graphene. In some cases, proposals are based on the concatenation of previously known monomers[8–10]. However, many of these structures are simply presented by the authors with no further explanation or attempt to organize them hierarchically[11–13]. This can be attributed to their intuition or trial-and-error approaches to find intricate ways to interconnect rings of different sizes in a periodic 2D tiling. In this context, a classification methodology

[1]Engineering Science and Mechanics Department, The Pennsylvania State University, State College, PA, USA. [2]Departamento de Física, Universidade Federal do Piauí, Teresina, Piauí, Brazil. e-mail: vincent.meunier@psu.edu



	V0	V1	V2	V3	V4	V5
V0	0	1	1	0	0	1
V1	1	0	1	0	1	0
V2	1	1	0	1	0	0
V3	0	0	1	0	1	1
V4	0	1	0	1	0	1
V5	1	0	0	1	1	0

Fig. 1 | Different atomic structures described by the same adjacency matrix. a Graph of the $r4^16^18^1$ system (where the 6 vertices are labeled V_i, $i = 0…5$) and **b** its corresponding adjacency matrix. **c**, **d** Graphs of the $h3^212^1$ and $c5^28^1$ systems, which are described by the same adjacency matrix as in (**b**).

was recently developed to allow for a more systematic way to account for the wide range of structures explored[14].

Another pathway to the proposal of new 2D carbon forms is the use of bottom-up search algorithms[15,16]. Certain approaches are specifically proposed and suited to the design of sp^2 carbon lattices, like topology-based searches for Haeckelite-like systems through the introduction of Stone-Wales (SW) defects[7]. The desired number of atoms in the unit cell is a commonly used variable in algorithms and packages based on evolutionary structural search[17]. Particle swarm optimization, symmetry constraints, and definitions of energy penalty functions are other tools used in the development of procedures dedicated to structure search[18,19], including variations in which porosity is a driving variable to the structural search[20].

A marked characteristic of these search engines is that they usually act in the space of coordinates, based on real variables. However, a given two-dimensional lattice can be uniquely identified by the collection of its polygonal rings, by knowing what rings touch each other, and how bonds are arranged. The problem thus becomes one of tiling periodic planes with polygons. In principle, the search for possible ways to tile 2D sp^2 carbon could be made simpler using integer variables. The foundational technique to explore this approach is that of graph theory.

Graph theory can be used to study how complex shapes are connected, such as covalently-bonded crystal structures[21]. At its most basic level, graph theory examines which vertices (or points) are connected by edges (or lines). In that sense, a covalently-bonded structure can be seen as a graph made up of atoms and bonds[14,21,22]. The adjacency matrix constitutes a basic way to describe a graph. However, as we shall discuss, there is not a one-to-one relation between the adjacency matrix and a two-dimensional, three-connected system.

Here, we introduce a graph theory-based tool to completely describe a structure's topology in terms of the atom-to-atom connections and the faces they form. This tool involves an integer-valued tensor that facilitates the systematic search of new geometries. We show that this tensor can be used to identify wallpaper symmetries in an exact way, without relying on tolerances for the computation of atomic distances, since no coordinates are used. Furthermore, when atomic positions are needed for further studies, we show that starting coordinates can be obtained straightforwardly, once the structures have been identified.

The paper is organized as follows. After establishing basic concepts, we define the tensor and associated *summation rules* to demonstrate how additional information on the structures can be obtained. We then show that the embedding tensor can be represented via the graph-theoretical concept of *flags*, a property that leads to the development of a coordinate-free symmetry determination. We then show how the tensor is a powerful tool to facilitate the systematic search for new structures and their classification. Finally, a rapid method to obtain approximate coordinates is presented and illustrated for a number of structures.

Results

The adjacency matrix

The adjacency matrix A (also known as the *connectivity table*) is a fundamental tool to store and study graphs[21]. It consists of a table in which there is a one-to-one correspondence between its rows/columns and the available vertices in the graph. The intersection of a particular row and column is marked with either a 1 or a 0, indicating if the corresponding vertices are linked or not, respectively. Explicitly, it is defined as:

$$A_{ij} = \begin{cases} 1, & \text{if vertex i is connected with vertex j, and} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Here, we focus on using graph theory tools in the context of periodic carbon sp^2 systems, so that the adjacency matrix is defined for the sites within the system's unit cell with periodic boundary conditions (PBC).

However, the adjacency matrix is not guaranteed to describe a unique crystalline system when we repeat the atomic enumerating scheme across periodic repetitions. This is illustrated with the examples in Fig. 1, which shows the edges and vertices within unit cells that create three different periodic structures with 6 atoms per primitive unit cell. The structures shown in Fig. 1a, c, d are labeled as $r4^16^18^1$ (biphenylene[23,24]), $h3^212^1$ (C3-12[25]) and $c5^28^1$ (popgraphene[26]), respectively, where the numbers in the sequence list the different rings present in the unit cell (3 for triangle, 4 for square, 5 for pentagon, 6 for hexagon, 8 for octagon, 12 for dodecagon), and the superscripts indicate the number of polygons of each type present within that cell, as described in ref. 14. In this notation, the front letter indicates the symmetry of the lattice system (e.g., r for rectangular, h for hexagonal, and c

Table 1 | Vertex-edge matrix of structures shown in Fig. 1

	E0	E1	E2	E3	E4	E5	E6	E7	E8
V0	1	0	0	0	0	1	1	0	0
V1	1	1	0	0	0	0	0	0	1
V2	0	1	1	0	0	0	1	0	0
V3	0	0	1	1	0	0	0	1	0
V4	0	0	0	1	1	0	0	0	1
V5	0	0	0	0	1	1	0	1	0

The six vertices are labeled V_i, $i = 0 \ldots 5$ and the nine edges are labeled E_i, $i = 0 \ldots 8$.

for centered). Examining the $r4^1 6^1 8^1$ system in Fig. 1a, one sees vertex 0 connected to vertices 1, 2, and 5. However, the $h3^2 12^1$ system shown in Fig. 1c features the same connections, as well as the $c5^2 8^1$ case shown in Fig. 1d. The same correspondence occurs for all the other sites of these structures. As a result, the adjacency matrices for the graphs illustrated in Fig. 1a, c, d are the same, corresponding to the table shown in Fig. 1b. While all the vertex-to-vertex connections are the same, the sets of faces they form are markedly different. A possible way to resolve this issue would be to introduce a complex adjacency matrix, where periodic images are endowed with a phase, in much the same way as implied by the Bloch theorem. However, this approach would require knowledge of the atomic coordinates of the structure, a requirement we wish to avoid for reasons that will become clear in this paper.

The adjacency matrix simply describes the connections between different vertices, as shown in Fig. 1b. An alternative, but equivalent, way of storing this information would be to describe the relationship between vertices and the edges they touch. Here, the connections would be described by a matrix in which the rows and columns would list vertices and edges, respectively, so that the matrix entries would contain either 0 or 1, depending on whether the edge contains the corresponding vertex (1) or not (0). Using Table 1 as an example, the entry in column 2, corresponding to an edge, and row 3, corresponding to a vertex, is 1, indicating that edge 2 contains vertex 3.

We can further expand the structure of this vertex-edge adjacency matrix to a more detailed tool. For this, we label each vertex, edge, and face (polygon) in a given network, and define a 3-index array to describe how the studied structure is constructed. In much the same way a two-dimensional table is a matrix, this three-dimensional table is a rank-3 *tensor*. As we will discuss below, this graph-theory-based approach allows us to characterize and understand atomic networks and their corresponding properties without requiring atomic coordinates. We illustrate this approach for the description of threefold-connected carbon nanosystems and introduce this tool as the *embedding tensor*, with the objective of characterizing their connectivity and the rings present in them[14]. This technique is a powerful generalization of previously known approaches, such as the adjacency matrix, since all properties related to the adjacency matrix can be inferred from the tensor. For instance, it is customary to see the adjacency matrix as a way to describe the covalent-bond network associated with the crystal structure. Properties such as the Estrada index are based on the eigenvalues of the adjacency matrix and therefore can also be derived from the embedding tensor.[27] However, the embedding tensor includes more information than the adjacency matrix, and uniquely describes each structure of interest here.

Classification theorem of a compact surface

Before formally introducing the embedding tensor, we recall some concepts in topology. The tensor we will introduce is used to describe 2D periodic carbon lattices. Here we will focus on full-sp^2 systems, in the sense that each carbon atom connects to exactly three other carbon atoms, though one stresses that its applicability is not limited to 3-fold connected carbon lattices. Such periodic 2D lattices are described in terms of a unit cell that can always be described in terms of a parallelogram-shaped unit. Topologically, such a unit cell can also be cast into a torus, as one can join two opposite edges of the cell to form a cylinder, and link the cylinder ends to form a torus.

To infer additional properties of the tensor, we need to understand how it reflects the properties of the underlying surface. For this, we turn to the *classification theorem* of compact surfaces, which states that the type of surface can be identified using only two pieces of information: the surface's rank or Euler characteristic, and its orientability. Using the theorem, we can state that the tensor can also describe the surface on which it sits.

Orientability is a property of a surface that allows for a consistent definition of *clockwise* and *counterclockwise* to exist. If there exists a path traversed that maps a clockwise-oriented object to its counterclockwise-oriented counterpart, the surface is non-orientable. Examples of non-orientable surfaces include the Klein bottle (periodic in two directions) and the Möbius strip (periodic in one direction). The structures we are interested in can be described as orientable surfaces, and they can always be mapped into a torus. It is the combination of the constraints set by Euler's characteristics *and* the orientability requirement that makes some periodic structures impossible to build, such as a primitive cell including only one pentagonal and one heptagonal ring. We note, however, that a non-orientable structure, though not physical in the present context, can also be described using the embedding tensor introduced in the next section.

The embedding tensor

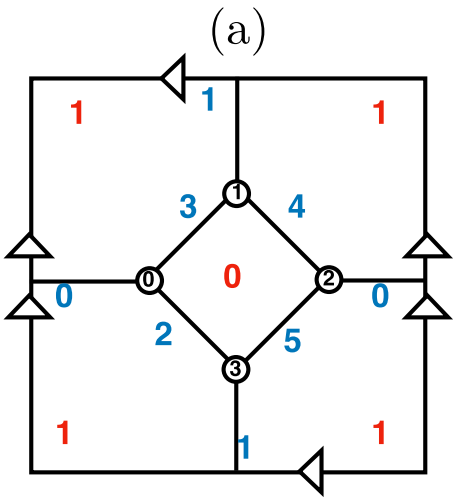
We now introduce a formalism to uniquely characterize a periodic 2D material. The *embedding tensor* Φ is a formalization of the framework to describe the type of crystalline systems shown in Fig. 1. We define it as follows. First, in this context, a face corresponds to a polygon. For example, a hexagon has six vertices and six edges. We can say that the face *contains* these vertices and edges. For instance, the entry of a rank-3 tensor Φ, at row 1, column 3, and depth 5, would identify the cell that relates to face 1, edge 3, and vertex 5. In this way, its entry will be 1 if face 1 includes edge 3 and vertex 5, or 0 otherwise. This leads to the general definition of the embedding tensor, Φ, as:

$$\Phi_{ijk} = \begin{cases} 1, & \text{if vertex k is contained in edge j} \\ & \text{and edge j is contained in face i, and} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Figure 2a illustrates a structure with two faces and shows its embedding tensor as two matrices representing its F_0 and F_1 frontal slices in Fig. 2b, c.

Φ is a rank-3 tensor. Specifically, it is a type of tensor known as a *symmetric tensor*. A symmetric tensor is invariant under permutation of the frontal, lateral, and horizontal 2D slices of the rank-3 tensor, as permutation is equivalent to relabeling the faces, edges, and vertices.

Note that the tensor can be written not only for the primitive cell (as in Fig. 2), but also for a conventional setup (as the 3×2 supercell illustrated in Fig. 3). Figure 3 shows how the tensor is notated as embedded on a surface. We can see faces labeled red, edges labeled blue, and vertices labeled black. For example, edge 20 connects vertices 10 and 14, and connects faces 8 and 9. So, the entries $\Phi_{8,20,10}$, $\Phi_{8,20,14}$, $\Phi_{9,20,10}$, and $\Phi_{9,20,14}$ will all be 1. In principle, as in the example above, each edge is associated with four non-zero entries of the tensor.



F_0	V_0	V_1	V_2	V_3
$E0$	0	0	0	0
$E1$	0	0	0	0
$E2$	1	0	0	1
$E3$	1	1	0	0
$E4$	0	1	1	0
$E5$	0	0	1	1

F_1	V_0	V_1	V_2	V_3
$E0$	1	0	1	0
$E1$	0	1	0	1
$E2$	1	0	0	1
$E3$	1	1	0	0
$E4$	0	1	1	0
$E5$	0	0	1	1

Fig. 2 | Illustration of the embedding tensor of 2D carbon nanostructures. a Graph of the $t4^18^1$ structure, with labeled faces, edges, and vertices shown in red, blue, and black, respectively. The structure is labeled according to the naming scheme described in ref. 14. The notation expresses that the unit cell is made up of one square and one octagon and is of tetragonal type ("*t*"). The embedding tensor of the structure is shown on the right. *E*, *V*, and *F* index the edge, vertex, and face, respectively. The depth of the tensor is 2 (2 faces). The tensor's frontal slices corresponding to faces F_0 and F_1 are shown in (**b**) and (**c**), respectively. All the sum rules described in the text are met.

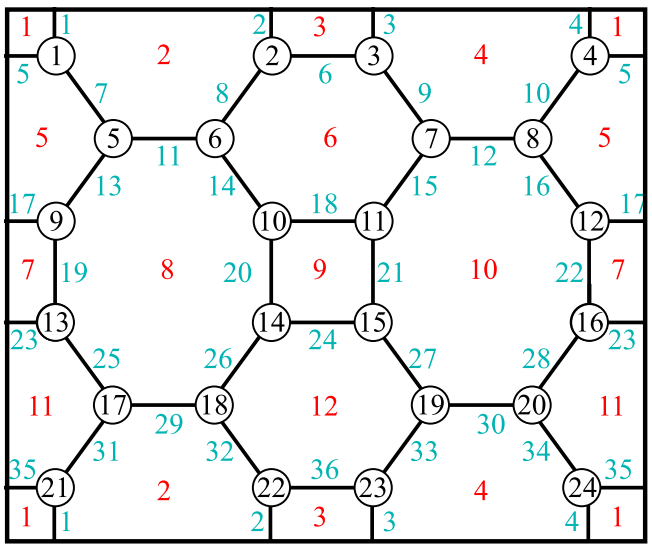


Fig. 3 | Illustration of how the different structural elements are labeled to define the embedding tensor in the case of a 3 × 2 supercell of a $r4^16^18^1$ system. Here, faces/edges/vertices are labeled by red/blue/black color numbers.

Furthermore, vertex 14 is part of edges 20, 24, and 26, as well as faces 8, 9, and 12, so that the entries $\Phi_{8,20,14}$, $\Phi_{8,26,14}$, $\Phi_{9,20,14}$, $\Phi_{9,24,14}$, $\Phi_{12,24,14}$, and $\Phi_{12,26,14}$ will all be 1. In contrast (in most cases), for 3-connected systems we expect six non-zero tensor entries to be associated with a given vertex of the structure. However, exceptions occur in certain cases, as we will discuss when introducing sum rules associated with the embedding tensor. Finally, let us look at the example of face 9. It is a square and contains vertices 10, 11, 14, and 15, as well as edges 18, 20, 21, and 24. So, the entries $\Phi_{9,18,10}$, $\Phi_{9,20,10}$, $\Phi_{9,18,11}$, $\Phi_{9,21,11}$, $\Phi_{9,20,14}$, $\Phi_{9,24,14}$, $\Phi_{9,21,15}$, and $\Phi_{9,24,15}$ will be non-zero. Usually, a face with *N* edges is expected to be associated with 2*N* non-zero entries of the tensor.

Summation rules

The networks have properties that make certain combinations of rings and connectivities impossible to exist. This is reflected in a number of summation rules.

First, we consider Euler's characteristics. The size of the tensor consists of three numbers, N_F, N_E, and N_V, which correspond to the number of faces, edges, and vertices, respectively. For a threefold connected, periodic system, we have[28]:

$$N_E = \frac{3N_V}{2}, \tag{3}$$

and

$$N_F = \frac{N_V}{2}. \tag{4}$$

We note that these equations require that the number of atoms ($N = N_V$) be an even integer. This basic property is at the core of the recursive *add-dimer* search algorithm discussed later. We stress that the rank-3 tensor is an integer tensor:

$$\Phi \in \mathbb{N}^{N_F \times N_E \times N_V}. \tag{5}$$

By computing $N_F - N_E + N_V$, we can easily verify the Euler characteristic χ, by

$$N_F - N_E + N_V = \chi = 0, \tag{6}$$

since, in the case of a periodic 3-connected 2D structure, $\chi = 0$ for a torus (for an orientable surface) and for a Klein bottle (for a non-orientable surface).

We now turn to the Face-Vertex rule, which consists of looking at a specific edge *j* and summing Φ_{ijk} over all faces and vertices, which usually gives

$$\sum_{i,k} \Phi_{ijk} = 4, \tag{7}$$

for each edge. This relationship reflects what we discussed before: an edge is associated with four non-zero entries of the tensor.

Some caution must be taken in some cases, in particular in smaller systems or in structures with large rings. In some of those special cases, the summation rule above does not hold, namely when the edge is a *bridge*. In graph theory vocabulary, a bridge is an edge that, if removed, splits a finite connected graph into two connected components. In the present context, we have to adapt this definition to the periodic systems we aim to describe. Here we are dealing with periodic 2D systems through the graph of the unit cell plus PBC. A bridge occurs when a polygon *borders* itself through a given edge. This is illustrated in Fig. 4a, where the edge linking vertices 1 and 2 (blue indices) is a bridge. In this example, the dashed red lines represent two parallel boundaries of the unit cell, and face 1 (red index) extends beyond the

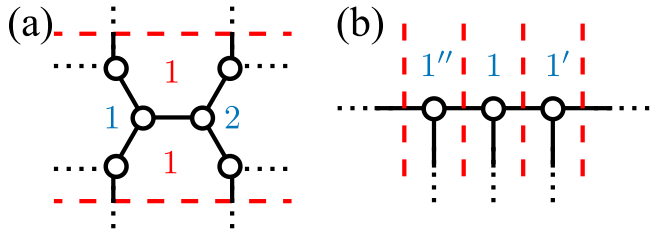


Fig. 4 | Bridge elements in an atomic network. a Illustration of a bridge, corresponding to the edge connecting atoms 1 and 2 (blue indices). Here, the bridge connects face 1 (red index) to itself through periodic boundary conditions. **b** Hypothetical structure featuring a loop, connecting an atom labeled as 1 to its periodic images labeled as 1′ and 1″. The dashed red lines in **a** and **b** represent the borders of the unit cell.

unit cell. Here, the periodic image of face 1, on the opposite end of the cell, connects to the original face through the edge linking vertices 1 and 2. Since a face and its periodic images are equivalent in the tensor description, we say the polygon touches itself, and the involved edge is defined as a bridge.

Bridges can be found in systems with sufficiently small unit cells and/or large polygonal rings. An example can be seen in Fig. 2, where both edges 0 and 1 are bridges. From the point of view of the whole 2D lattice, removing a bridge (and its periodic images) splits the system into an infinite number of (quasi-1D) connected components. However, from the point of view of the unit cell graph with PBC, removing a single bridge does not necessarily divide the graph into two connected components. For example, when removing either edge 0 or edge 1 in the case shown in Fig. 2, the unit cell graph with PBC is still composed of a single connected component. However, for simplicity, we will still use the term *bridge* for such cases. When edge j is a bridge, we have:

$$\sum_{i,k} \Phi_{ijk} = 2, \tag{8}$$

since we have only 2 tensor entries associated with bridge j (as the same face touches both sides of the bridge). It follows that the presence of a bridge can be straightforwardly established by calculating $\sum_{i,k}\Phi_{ijk}$.

Another case should be considered, the one involving *loops*. In graph theory, a loop is an edge that connects a vertex to itself. For the periodic PBC graph we are dealing with, a loop is an edge that connects a vertex (an atom) to one of its periodic images, with the loop necessarily crossing the edge of the unit cell. This is illustrated for the atom labeled as 1 in Fig. 4b, which is connected through a loop to its image labeled as 1′. Here, the loop crosses the cell edge represented by the dashed red line. In contrast, atom 1 also has to be loop-connected to its image labeled 1″ in Fig. 4b. The 1-to-1′ and 1-to-1″ edges end up being parallel, and the periodicity produces a structure with successive T-like sets of atoms and edges, which prevents us from building a 2D lattice. For this reason, loops are not possible for the 2D PBC graphs of periodic 3-connected systems.

Turning to the Face-Edge rule, assuming that there exist no bridges or loops in the tensor, the sum of Φ_{ijk} over i and j is twice the number $V(k)$ of connections that vertex k has:

$$\sum_{i,j} \Phi_{ijk} = 2 \times V(k), \tag{9}$$

for each vertex. In the context of 3-connected structures, $V(k) = 3$ for all sites, and the result of the sum rule should be 6. This simply reflects the fact discussed before: that (usually) there are six non-zero entries of the tensor associated with a given vertex. In case the vertex is connected to a bridge, this result has to be subtracted by one unit for each emerging bridge, since both sides of such an edge correspond to the same face. We also note that the number of bridges emerging from a vertex can never be 2. If a single bridge emerges from a vertex, two of the three faces touching the vertex are the same. If there is a second bridge on the same vertex, it necessarily involves the looped face and the remaining one, so that all three faces touching the vertex are the same, and all three emerging edges are bridges. In this way, the result of the Face-Edge rule for a given vertex in a 3-connected structure is either 6 (no bridges), 5 (one bridge), or 3 (three bridges).

For the Edge-Vertex rule, the sum of Φ_{ijk} over j and k is twice the number $F(i)$ of edges each polygon i has:

$$\sum_{j,k} \Phi_{ijk} = 2 \times F(i), \tag{10}$$

for each face i. This can be derived from the previous rule and the operation swapping faces and vertices, or it can be motivated geometrically by showing that a polygon with N_E edges contains $2N_E$ non-zero tensor elements. Each edge has four non-zero tensor entries, 2 per face touching the edge. Therefore, as each edge donates two per face, a face with $F(i)$ edges has $2F(i)$ non-zero tensor entries. Notably, this rule gives direct access to the make-up of the crystal in terms of polygon size and their number.

However, this rule changes in the presence of bridges. For a bridge (say, edge e), we can associate two vertices (v_1 and v_2, for instance), but a single face (say, face f), since this face touches itself through the bridge). So, there are not four, but only two non-zero tensor entries associated with the bridge: Φ_{f,e,v_1} and Φ_{f,e,v_2}. Hence, since a bridge works simultaneously as two sides of the polygonal face touching it, we have to discount the above result by 2 for each bridge composing the face.

There are also a number of matrix reduction rules. Suppose we focus on a given edge-vertex pair, represented by the j (edge) and k (vertex) indices in the tensor entry Φ_{ijk}, for a given threefold-connected tiling. Summing up these elements over the faces results in

$$\sum_{i} \Phi_{ijk} = \begin{cases} 2, & \text{if edge j contains vertex k, and} \\ 0 & \text{otherwise.} \end{cases} \tag{11}$$

The first case results from the fact that edge j borders two faces, and both contain vertex k. This sum can be seen as a matrix M with elements $M_{j,k} = \sum_i \Phi_{ijk}$, where each row (column) represents an edge (a vertex). For this reason, we refer to this as a matrix reduction rule. In the case we are dealing with an edge that is a bridge, the corresponding non-zero $M_{j,k}$ elements will be 1 instead.

This rule can also be applied to the case in which we focus on a given face-vertex pair, represented by the i and k indices in the tensor element Φ_{ijk}. Summing up over the edges results in

$$\sum_{j} \Phi_{ijk} = \begin{cases} 2, & \text{if face i contains vertex k, and} \\ 0 & \text{otherwise.} \end{cases} \tag{12}$$

This results from a vertex and a face sharing two entries, defined by the edges that border both of them. This comes from the fact that only two of three edges containing site k are also connected to face i if this face contains the vertex i. On the other hand, if one of these two edges is a bridge, the third edge emerging from k will also be part of the face i, as the face touches itself through the bridge, so the resulting $\sum_j \Phi_{ijk}$ will be 3.

For completeness, we can also write the rule considering a face-edge pair and summing over the vertices:

$$\sum_{k} \Phi_{ijk} = \begin{cases} 2, & \text{if face i contains edge j, and} \\ 0 & \text{otherwise.} \end{cases} \tag{13}$$

Finally, according to the total sum rule, the sum of all tensor elements is

$$\sum_{i,j,k} \Phi_{ijk} = 4N_E. \tag{14}$$

This rule implicitly considers the system with no bridges. So, the total sum rule simply reflects the fact that there are four non-zero entries per edge.

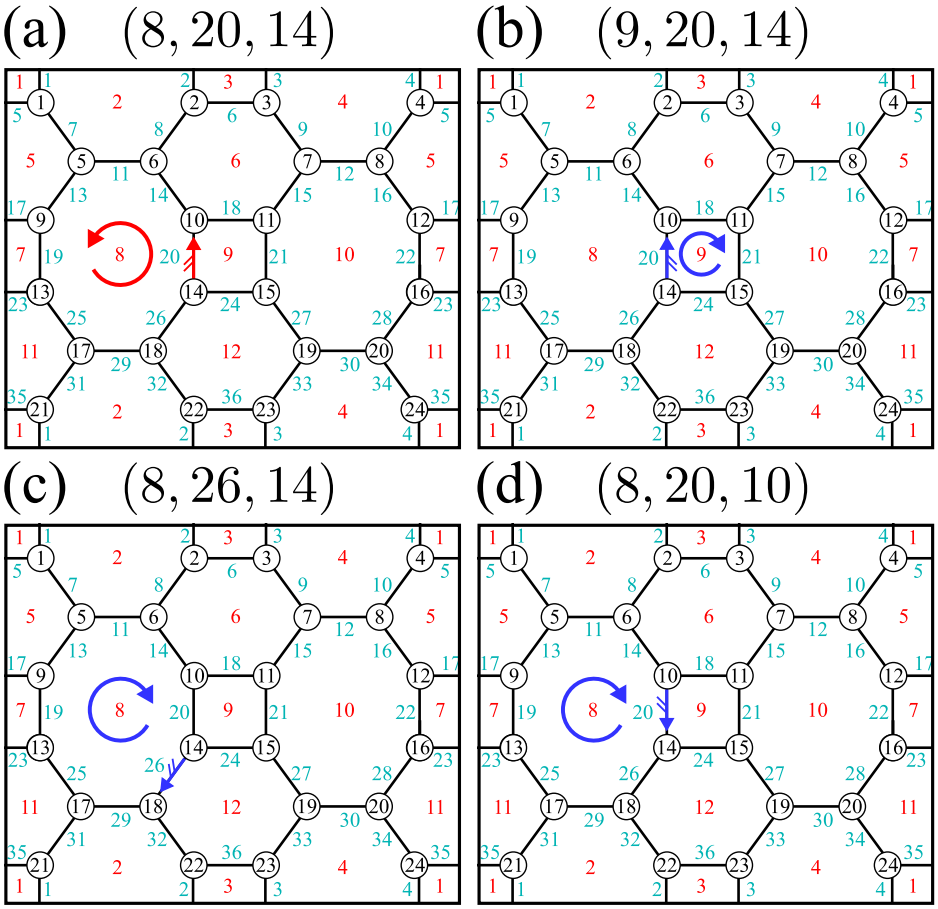


Fig. 5 | Graph of a 3 × 2 supercell of the 3-connected $r4^1 6^1 8^1$ structure, with labeled faces (in red), edges (in blue), and vertices (in black). **a** Illustration of the (8,20,14) flag (red line), with its counter-clockwise orientation indicated by the curved red arrow in the center of the octagonal ring labeled as face 8. **b** The (9,20,14) flag (neighboring the (8,20,14) one) represented by a blue arrow, together with its clockwise orientation indicated by the curved blue arrow in the center of the square ring labeled as face 9. **c** Same as **b**, but for the (8,26,14) flag (blue arrow) with an indication of its clockwise orientation (curved blue arrow in the center of face 8). **d** Same as **b**, but for the (8,20,10) flag (blue arrow) with an indication of its clockwise orientation (curved blue arrow in the center of face 8).

The presence of a bridge in the system reduces the result of the total sum rule by 2, as there are only 2 (not 4) non-zero entries associated with it within the tensor. In this way, $(4N_E - \sum_{i,j,k}\Phi_{ijk})/2$ gives the number of bridges in the system.

Flags

In much the same way the adjacency matrix is composed of vertex-vertex connections, the embedding tensor is made out of entities called *flags*. Flags are concepts of graph theory, and they are defined by the 3-dimensional list of polytopes in the form (f, e, v), with f for face, e for edge, and v for vertex. An alternative way to assemble the embedding tensor from flags is that

$$\Phi_{ijk} = \begin{cases} 1, & \text{if flag(i, j, k)exists in the polytope, and} \\ 0, & \text{otherwise.} \end{cases} \quad (15)$$

We suggest a systematic way to visualize a (f, e, v) flag over a 2D carbon structure using a graphical element. It consists of an arrow with one of its sides marked by a pair of parallel straight lines. The arrow is drawn over edge e, so as to emerge from vertex v, and the pair of lateral parallel straight lines is placed on the side of face f. An example is shown in Fig. 5a, with the (8, 20, 14) flag represented by a red arrow. This graphical representation of flags makes it easier to visualize the orientability of the structure. From the point of view of the flag's face f, the arrow representing the flag can be associated with a clockwise or counterclockwise rotation around the face's center. In the example shown in Fig. 5a, the (8, 20, 14) flag can be associated with a counterclockwise rotation around the center of the corresponding face, represented by the curved red arrow.

We can also define the neighbors of a flag. These neighbors are the same as the reference flag, except for one of its components. Namely, the (f, e, v) flag has three neighboring flags, each of the (f', e, v), (f, e', v), and (f, e, v') forms. Here f' borders f, while e and e' both emerge from the same vertex v, and v' belongs to the same edge e which contains v. To easily identify the nature of the neighboring flags connections, we will refer to the (f, e, v) and (f', e, v) pairs as *face-neighbor flags*, (f, e, v) and (f, e', v) as *edge-neighbor flags*, and finally (f, e, v) and (f, e, v') pairs as *vertex-neighbor flags*. For the structure shown in Fig. 5a, the face-, edge-, and vertex-neighbors of the (8, 20, 14) flag are illustrated in Fig. 5b–d by blue arrows. One should note that while (8, 20, 14) is a counterclockwise flag, its three neighboring flags have a clockwise orientation (as indicated by the curved blue arrows in Fig. 5b–d). Likewise, in an orientable structure, the neighbors of any flag are flags of the opposite orientation.

With these concepts in mind, we can define a graph where the flags play the role of vertices, and the edges correspond to connections between pairs of neighboring flags. Then, the orientability of a surface can be determined with the aid of the flag graph, through a search within the embedding tensor. In an orientable tensor, the flags and their operations form a bipartite graph, with local clockwise and counterclockwise orientations alternating for neighboring flags. For instance, in Fig. 6a–f we illustrate the flag graph sector corresponding to the six flags emerging from vertex 4 of a given structure. Flags (3, 1, 4) and (2, 1, 4) are face-neighbors, as are (3, 2, 4) and (1, 2, 4), and the (1, 3, 4) and (2, 3, 4) pairs. Similarly, the (3, 1, 4) and (3, 2, 4) flags are edge-neighbors, similar to (2, 3, 4) and (2, 1, 4), as well as (1, 2, 4) and (1, 3, 4). In the diagram shown in Fig. 6g, we represent the six flags associated with vertex 4 by circles connected to form a hexagon (see central hexagon in Fig. 6g). Clockwise (resp. counterclockwise) flags are indicated in blue (resp. red) color. In this hexagon, the green (resp. red) edges represent connections between face-neighbor (resp. edge-neighbor) flags. This hexagonal unit of the flag graph is directly related to our discussion on the face-edge sum rule. So, this hexagon from the flag graph connects to the hexagons representing the flags of the other 1, 2, and 3 vertices through squares, as illustrated in

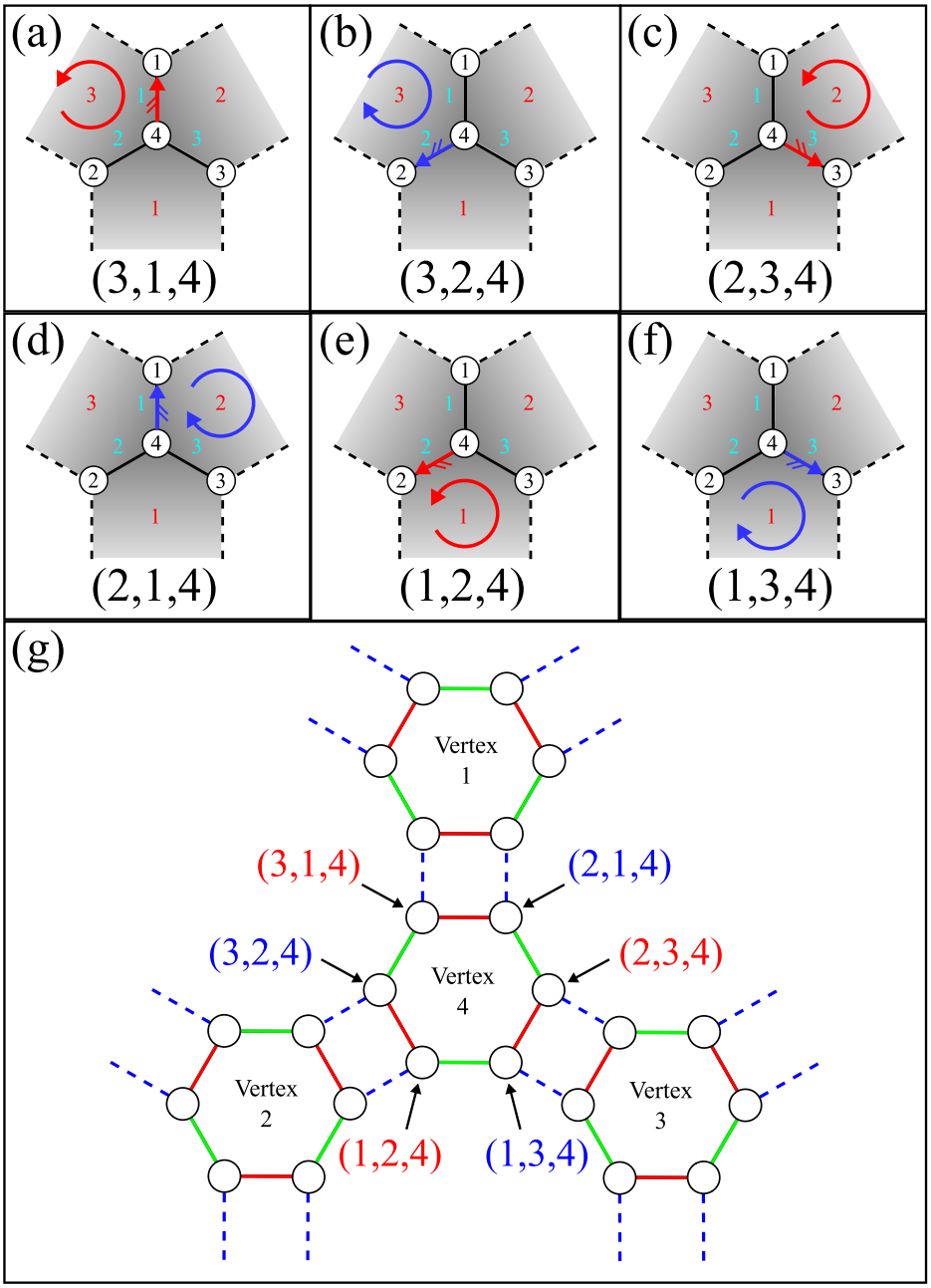


Fig. 6 | Representation of flags and connections. **a**–**f** Representation of the flags emerging from site 4 of a given structure. Red numbers label faces, cyan numbers label edges, and black numbers label vertices, while counterclockwise and clockwise flags are represented in red and blue, respectively. **g** Representation of the local connections of the flag graph for the flags emerging from site 4 and their connections with those flags emerging from neighboring vertices.

Fig. 6g. The dashed blue connections link vertex-neighboring flags. This square unit of the flag graph is related to the fact that four flags are associated with each (non-bridge) edge, as also illustrated by the face-vertex sum rule. This results in a bipartite graph, as we have only even-membered rings and an orientable surface. Note that, in addition to the hexagons and squares represented in Fig. 6g, any N-membered ring in the original graph becomes a $2N$-ring in the flag graph.

An easy way to understand the overall form of a flag graph, in terms of the original graph defining the studied system, is by looking at the relationship between graphene and graphenylene. The graphenylene network can be conceptually thought of as (1) starting with graphene and (2) substituting each carbon site by a hexagonal ring and substituting each bond by a square ring[29]. In other words, graphenylene consists of hexagonal rings connected to each other by squares, with the hexagons organized in a honeycomb-like lattice, just as carbon atoms in graphene. While graphene and graphenylene are two different structures, the flag graph of graphene is analogous to a graphenylene network, where each site now represents a flag. We can follow a similar procedure and construct the flag graph of any periodic 3-connected structure and substitute each atom by a hexagonal ring, which is further connected to other hexagons by squares (where before we had an edge), with each site of this new graph being a flag.

Figure 7 shows the periodic representation of the set of all possible structures with 2, 4, and 6 atoms per unit cell, together with their corresponding primitive cells indicated by the red colored cells. Here, each circle represents a carbon atom, and the lines indicate pairs of bonded atoms. In each case, we also illustrate the corresponding flag graph represented over their primitive cell. In these flag graphs, each circle represents a flag, while face, edge, and vertex-neighbor connections are represented by red, green, and blue lines. The bipartition of these lattices is highlighted by the alternating black and white shading for neighboring circles.

In non-orientable graphs, bipartition is locally preserved, but globally broken, as there exists a path that reverses orientation, by definition. Therefore, on non-orientable tensors, the flag graph is no longer bipartite.

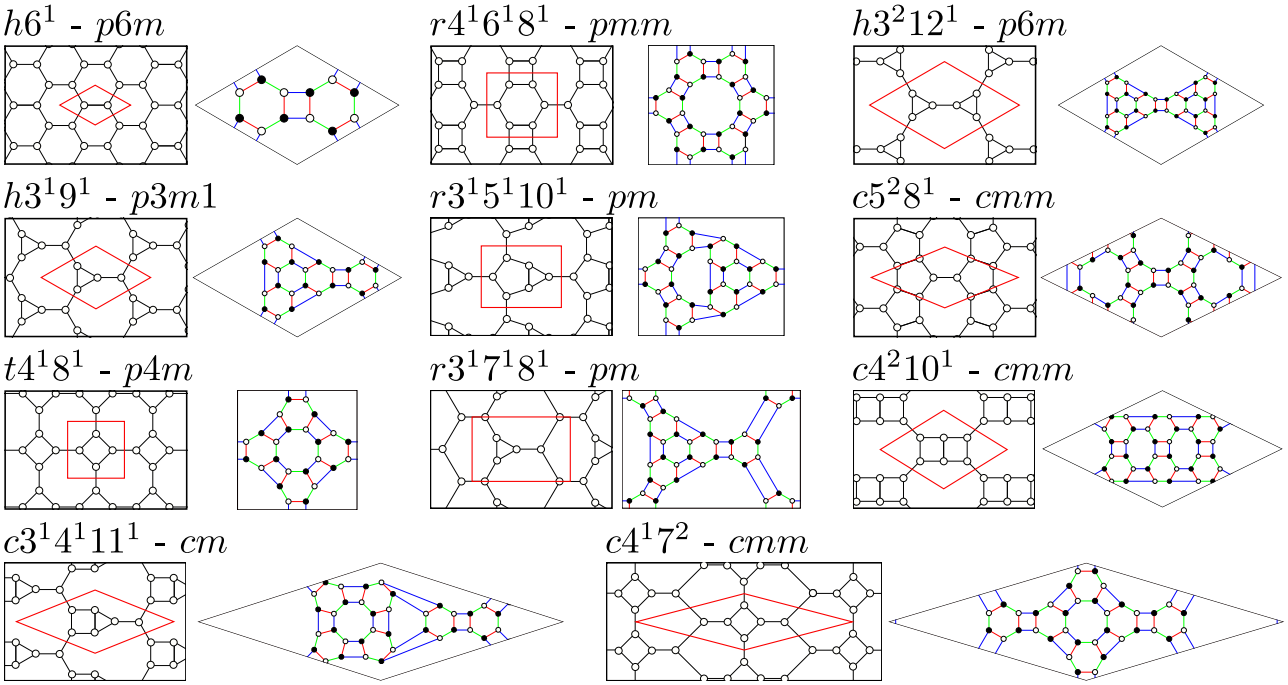


Fig. 7 | Representation of different systems with 2, 4, and 6 atoms per primitive cell, with their primitive cell indicated in red. We show a representation of the flag graph of each structure over the primitive cell. Face, edge, and vertex-neighbor connections in the flag graphs are represented by red, green, and blue color lines, with the alternating black and white color for the circles illustrating graph bipartition. Those structures are therefore orientable with PBC corresponding to the topology of a torus.

We can test for orientability by alternating the coloring of neighboring flags until either all flags are colored or two neighboring flags are forced to have the same color. Those aspects related to the bipartition of the flag graph in orientable graphs will be used in the symmetry section to analyze whether a certain symmetry is orientation-preserving (e.g., a rotation) or not (e.g., a mirror or glide reflection).

Neighbor flags are also useful to define how we *walk* over the structure through the flag graph. When going from one flag to another, we can move from the first one to either its face-, edge-, or vertex-neighbor flag. Such moves are defined as the result of the application of the $\widehat{\sigma}_f$, $\widehat{\sigma}_e$, and $\widehat{\sigma}_v$ operators, respectively. So, the order of *f*-, *e*-, and *v*-flips properly defines a walk, which will be an important tool for symmetry search, as we will discuss now.

Symmetry

We will now show how the embedding tensor can be used to determine the maximal symmetry a structure can have, without needing any information on the coordinates of the vertices or its lattice vectors. For this reason, this method does not require a tolerance criterion as coordinate-based symmetry approaches assume. This is especially useful in the context of the high-throughput, systematic production of 2D tilings, as introduced in the Appendix of ref. 14 or in the recent paper of Chen[7], for example. We will revisit the search procedure, and show how it benefits from the power of the embedding tensor, in Section "Structural search algorithm".

We will now focus on the concept of symmetry maps. First, let us consider two embedding tensors, Φ_A and Φ_B, and examine the conditions under which they describe the same surface. Here, *being the same* means we can find an isomorphism relation between the tensors Φ_A and Φ_B. Then there must be a one-to-one correspondence from the faces, edges, and vertices in Φ_A to respective faces, edges, and vertices in Φ_B. Since a flag can be described with a set of faces, edges, and vertices, and those correspondences are well defined, a one-to-one correspondence must exist between the flags of the two tensors. In this case, we say there is a *map* between the flags of the two tensors. For these reasons, we will focus our discussion on the map between the flags. Before moving on, we need to clarify that when we say that two tensors are identical, we always assume that they are identical *modulo* the permutation (or renumbering) of faces, edges, and vertices. To avoid possible confusion, we will use the sign " ≡ " to emphasize this point.

Suppose we are comparing a tensor Φ with a tensor resulting from the application of a *symmetry operation*: $\Phi' = \widehat{S}\Phi$. A symmetry exists in the structure if the operator $\widehat{S}$ maps an embedding tensor Φ onto itself ($\widehat{S}\Phi \equiv \Phi$). The symmetry operators then permute the faces, edges, and vertices, while leaving the underlying structure unchanged, so the equation is similar to an eigenvalue problem.

The symmetry operator $\widehat{S}$ can also be viewed as operating on the surface's flags composing Φ. Since the flags are stored in the form of (f, e, v), in general, we have $\widehat{S}(f, e, v) = (f', e', v')$ where the symmetry operator takes one flag of Φ to a flag of Φ'. But we can go further: if $\widehat{S}(f, e, v) = (f', e', v')$, then $\widehat{S}f = f'$, so one face in Φ will be consistently mapped to f', and similarly for edges and vertices. If we repeat this procedure for each face, edge, and vertex, then we will have the full map, including every flag in Φ mapped to a flag in $\Phi' = \widehat{S}\Phi$.

However, we do not know the symmetry operations of the object beforehand and we have to search for them. In reality, we search for *maps* that can be later associated with symmetry operations. In order to search for these maps, we look within the tensor for two flags, (f, e, v) and (f', e', v'), that have the same size of face and local connections. Then, we change our point of view and interpret this pair of starting flags as one coming from Φ and the other from $\widehat{S}\Phi$, and suppose they are equivalent under a symmetry operation. Then, a symmetry flag map can be tested by walking over the set of flags of the Φ tensor starting from (f, e, v), at the time we perform the same walk starting from (f', e', v') (for the symmetry-transformed $\widehat{S}\Phi$ tensor). The *walk* is defined by the sequence of $\widehat{\sigma}_f$, $\widehat{\sigma}_e$, or $\widehat{\sigma}_v$ walking operators used to scan the structure by swapping a face, an edge, and a vertex, respectively. For each step, we check for local symmetry. In principle, the one-to-one flag correspondence derived from these walks describes the entirety of that symmetry map, with every flag having a pairing under such a map. If such a map exists, the Φ and $\widehat{S}\Phi$ tensors are said to be equivalent under symmetry. If an error appears in the algorithm due to mismatched local structure, the

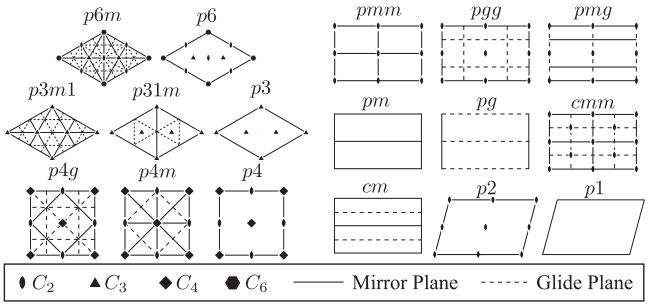


Fig. 8 | Schematic representation of the 17 wallpaper groups, the possible symmetry groups in two dimensions.

original assumption that there is a symmetry map between (f, e, v) and (f', e', v') is incorrect, and $\widehat{S}$ is not a symmetry operation. Mismatched structures can occur because matched faces have different numbers of edges. In principle, this framework can be extended to systems not represented by 3-connected graphs, so that mismatches may occur when vertices have different degrees. Finally, we extend these tools for systems containing more than one chemical species, so that mismatches will also occur when vertices represent different atom types. To check this in an efficient manner, we can store the relevant information in additional vectors, one vector describing the number of edges per face and another one describing the degree of each vertex. Moving beyond carbon, we can also include a vector describing the type of atom corresponding to each vertex. We also note that the present work already opens the possibility of creating BN or even BCN structures, as the vector storing the atom data could be used in the same manner as the vertex or face vectors to find potential mismatches.

To identify all the symmetry maps of the object, we choose one flag in tensor Φ and iterate over all flags of the same tensor, testing each for a symmetry map. Therefore, the algorithm's run-time is the square of the number of flags. As each edge corresponds to four flags in the 2D case, and the number of flags of a surface is $4 \times N_E$, the runtime of the algorithm is roughly $\sim 16N_E^2$, or $O(N_E^2)$. However, this operation is not computationally too taxing as all the processes involve integer arithmetic in fairly small numerical objects.

With the *proviso* of the previous discussion, we can now state that *two tensors are isomorphic if there exists an invertible map from one tensor to another*, since the problems of tensor isomorphism and tensor symmetry are intricately linked. Finally, the set of possible symmetries then forms the symmetry group of the object.

Restricting ourselves to periodic systems on a 2D surface, the symmetry groups of interest are the wallpaper groups. They consist of all the possible ways a 2D translationally invariant structure (in two different directions) can be symmetric. As a result, 17 different groups are possible to categorize the possible types of two-dimensional crystal structures[30], as schematically represented in Fig. 8. In most cases shown in Fig. 8 (i.e., group names starting with "p"), the symmetry elements are represented over the primitive cell. Exceptions are the *cm* and *cmm* groups, which are represented over conventional rectangular unit cells.

Usually, coordinate methods are used along with matrix transformations to find true symmetries, as with the SPGLIB library[31]. Using the symmetry list, atoms can be sorted into equivalence classes, where each class represents an atom plus symmetry. The idea of using Φ is that symmetries can be calculated in a way that does not involve coordinate-based methods with tolerances. While the tensor approach does not account for symmetry lowering due to certain distortion phenomena, such as those induced by Peierls and Jahn-Teller instabilities, or out-of-plane corrugation (see the case of TPH-β[32]), it allows for systematic search procedures to be run in a much simpler (integer numbers) space than with actual coordinates. In that sense, the algorithm used here provides the maximal symmetry a structure can possess.

Table 2 | Summary to identify the nature of the symmetry map through the maximal number of fixed elements ($n^{\max}_{fix}$) and orientation

Symmetry nature	n^{max}_{fix}	Orientation
Identity	3	+
Mirror	2	−
Rotation	1	+
Glide	0	−
Translation	0	+

Before presenting a detailed algorithm to extract the wallpaper symmetry group from the Φ tensor, we highlight that more information can be extracted during the generation of symmetry maps. For instance, if the surface is asserted to be orientable, then the symmetry parity can be recovered, allowing one to know whether the resulting symmetry preserves or reverses orientation. If a flag is mapped to another with the opposite orientation, then the whole symmetry is orientation-reversing (marked by a − sign) and is a mirror symmetry or a glide reflection. Similarly, if the flag is mapped to another one with the same orientation, the symmetry map preserves orientation (marked by a + sign) and is an identity, rotation, or translation symmetry.

Another important piece of information is the maximal number of fixed elements ($n^{\max}_{\text{fixed}}$) for a given flag transformed under the symmetry map. This would be done by running a tally of the maximal number of matched elements. For example, if (f, e, v) = (1, 2, 3) maps to ($f\prime$, $e\prime$, $v\prime$) = (1, 2, 4), then two elements (face and edge) are fixed under this mapping ($n_{\text{fixed}} = 2$), while a single element (the face) is fixed if (f, e, v) = (1, 2, 3) maps to ($f\prime$, $e\prime$, $v\prime$) = (1, 5, 4) ($n_{fixed} = 1$). Even though some (f, e, v) ↔ ($f\prime$, $e\prime$, $v\prime$) correspondences under a given $\widehat{S}$ operation may feature no fixed elements, other such correspondences may conserve $n_{fixed} = 1$, 2 or 3 elements, with the maximal n^{max}_{fixed} value of a given map being fundamental to identify the nature of the symmetry operator.

With glide reflections and translations, no items are fixed ($n^{\max}_{\text{fixed}} = 0$). Under a rotation, one item is fixed at the rotation center ($n^{\max}_{\text{fixed}} = 1$). Under a reflection, flags on the mirror plane are fixed, leading to $n^{\max}_{\text{fixed}} = 2$ fixed items. Finally, $n^{\max}_{\text{fixed}} = 3$ elements are fixed if and only if the symmetry is the identity. Regarding orientation, it is preserved (+) for rotations, translations, and the identity, while it is inverted (−) for mirrors and glide planes. So, combining $n^{\max}_{\text{fixed}}$ and the orientation profile allows us to identify the nature of the symmetry map, as summarized in Table 2.

We will now turn to the description of a symmetry search algorithm. In two-dimensional systems, there are rotations, mirror and glide planes, and translational symmetries, as well as the identity symmetry. The embedding tensor produces a flag graph that can be used to count symmetry maps by a systematic procedure. For each symmetry map, we can list whether it preserves or flips orientation, and the maximum number of fixed flag components. Following from previous sections, that would be enough to determine every type of symmetry map.

Here we will present a robust algorithm to identify the symmetry group out of the embedding tensor and the flag graph. If we go through all 17 wallpaper groups, inspection tells us that the expected number of rotation (N_{rot}), mirror (N_{mir}), and glide (N_{gli}) maps for a primitive cell are those listed on Table 3. To avoid complications related to the presence of bridges, we can assume a conventional 2 × 2 supercell to define the tensor, as no bridge or loop can exist in this supercell. In principle, it is possible to avoid using a supercell by introducing the concept of *twin-flags* when dealing with the presence of bridges. Here, we will avoid this complication and explain the concept of twin-flag in the Supplementary Information document. However, we highlight that the wallpaper group identified in the procedure outlined here corresponds to that of the primitive cell. We also note that when dealing with a non-primitive unit cell, maps resulting from the combination of a mirror with a translation end up being misinterpreted as glide maps. To cope with this in the symmetry identification approach,

Table 3 | Expected number of each type of symmetry map in a primitive cell in each wallpaper group

Group	Rotations	Mirrors	Glide
p1	0	0	0
p2	1	0	0
pm	0	1	0
pg	0	0	1
cm	0	1	0
pmm	1	2	0
pmg	1	1	1
pgg	1	0	2
cmm	1	2	0
p4	3	0	0
p4m	3	4	0
p4g	3	2	2
p3	2	0	0
p31m	2	3	0
p3m1	2	3	0
p6	5	0	0
p6m	5	6	0

mirrors and glides have to be counted together as one unit, so their total number is well defined for each group. By counting the number of symmetry maps of each type, we can determine which wallpaper group the structure belongs to most of the time. Namely, we determine N_{rot}, $N_{\mathrm{mir}} + N_{\mathrm{gli}}$, and the number N_{tra} of translation maps. We can then infer the number of rotation and mirror+glide maps in the primitive cell by dividing them by the size of the translation group, which is $N_{\mathrm{tra}} + 1$, due to the identity symmetry.

To distinguish the C_n ($p1$, $p2$, $p3$, $p4$, $p6$) groups from the D_n (everything else) groups, we can check if there is an orientation-flipping symmetry ($N_{\mathrm{mir}} + N_{\mathrm{gli}} \neq 0$). The next step is to determine the number of rotation maps per unit cell: $\frac{N_{rot}}{N_{tra}+1}$. Combining the $N_{\mathrm{mir}} + N_{\mathrm{gli}}$ and $\frac{N_{rot}}{N_{tra}+1}$ values, we are able to break the wallpaper groups into the following classes: ($p1$),($p2$),($p3$),($p4$),($p6$),(pm, cm, pg), (pmg, pgg, pmm, cmm), ($p3m1$, $p31m$), ($p4g$, $p4m$), and ($p6m$). The existence ($N_{\mathrm{mir}} \neq 0$) or not ($N_{\mathrm{mir}} = 0$) of mirror maps allows us to split (pg) from (pm, cm), as well as (pgg) from (pmg, pmm, cmm). This leaves us with 4 sets of wallpaper groups left to discern - (pm, cm), (pmg, pmm, cmm), ($p3m1$, $p31m$), and ($p4g$, $p4m$).

We need to gain more symmetry information to discern the remaining types. Using the same methods as calculating the size of the rotation group, we can label and count the number of unique fixed elements in each rotation. We can retrieve a list of unique elements by identifying two elements if there is any symmetry linking them. For example, the wallpapers $p3m1$ and $p31m$ have three rotation axes and three mirrors, but are arranged differently. In the case of $p31m$, the two rotation centers not on a mirror are equivalent by the mirror that separates them, whereas in $p3m1$ they are distinct. This process is able to take care of all of the possible results with the exception of pm and cm. Important data from this discussion can be found on Table 3.

Differentiating between pm and cm is considerably more complex. The main difference between the two groups is the number of distinct mirror lines. While pm has two, cm has one. However, both pm planes are described by a single map. To tell the mirror planes apart, we must have a way of counting the lines. We can use a technique similar to the rotation case—by finding items fixed by a mirror symmetry that are unique under all symmetries. Although, in this case, translations and mirrors are the only symmetries of note. The pm mirror symmetry map has two sets of fixed elements, but those two sets cannot be mapped to each other under a translation. Testing for this separates pm and cm.

To summarize the procedure, the algorithm can be run as follows:

(1) Read the Φ tensor and verify the sum rules;
(2) Extract the flags and create the flag graph by identifying their neighbors;
(3) Determine the flags' orientations according to the bipartition of the flag graph;
(4) Use the flag graph to search for symmetry maps and identify their nature (rotation, mirror, glide,…);
(5) Determine N_{rot}, N_{mir}, N_{gli}, and the size of the translation, rotation, and mirror groups, along with unique rotation centers and mirror planes;
(6) In special cases, determine if mirror symmetries exist (pgg,pg);
(7) Use the results to get the symmetry group.

Structural search algorithm

The embedding tensor provides a precious tool to facilitate the search for carbon allotropes. For instance, using an add-dimer algorithm, where carbon pairs are added to known structures to *grow* the structure from a common seed (i.e., graphene with its minimal 2-atom per cell crystal), the use of the embedding tensor requires only integer operations on a small array, making the search computationally efficient. Furthermore, the tensor can be used (1) to avoid equivalent structures by rejecting those whose tensor is equivalent (that is: identical modulo relabeling of face, edge, and vertices) to already identified ones, and (2) to identify non-primitive cells by determining whether the unit cell possesses translational symmetry via the symmetry algorithm. Recall that the adjacency matrix alone is insufficient to distinguish certain sets of structures, like those represented in Fig. 1, which have the same adjacency matrix. On the other hand, the embedding tensor allows us to easily distinguish if two structures are the same or not simply by verifying if there is at least one symmetry map between them.

Moreover, the tensor, as the output of the search, provides direct access to constructing the name of the allotrope, as described in ref. 14, making it possible to build metadata during a high-throughput search. What's more, the rapid access to the wallpaper symmetry during the search enables targeted exploration of structures with a desired symmetry.

To understand the efficiency of the search, recall that the add-dimer method explores all possible ways of inserting a pair of connected atoms into an existing structure. In principle, all pairs of existing bonds in a given face must be considered as potential insertion sites. This number grows rapidly and renders the search computationally prohibitive for even modest values such as $N_F \sim 8$. This is precisely where symmetry becomes especially useful: it allows one to restrict the exploration to symmetry-unique faces and symmetry-unique bonds within them. Figure 9 illustrates the embedding tensor-based add-dimer algorithm for $N_F = 1$ to $N_F = 2$ and $N_F = 2$ to $N_F = 3$. There, we represent the initial $N_F = 1$ parent structure, how it generates the three $N_F = 2$ (one of them being a conventional cell of the $N_F = 1$ system), and how these last three structures generate the set of $N_F = 3$ systems.

We provide extensive examples of how tensors are generated during the search for $N_F = 1, 2, \ldots 7$ in the online Dataset associated with this paper[33]. To illustrate the obtained results and the power of embedding tensor-based add-dimer algorithm, we provide statistical information, including symmetry, for systems with $N_F = 1, \ldots, 7$ faces in Table 4. We note that low-symmetry structures are the most common ones, with the $p1$ group featuring the largest relative number of structures in total, even though there are more structures of other symmetry groups for systems with up to 5 faces. Furthermore, despite some non-monotonic trends for small sizes, the number of structures increases in a nonlinear way, with the structures with 7 faces experiencing a huge increase relative to the 6 faces case. In particular, the total number of structures with 7 faces is more than three times the number of structures with 6 or fewer faces. Because triangles deviate too much from sp^2 bonding, we separate their count in the table for clarity. The number of systems containing triangles is remarkably larger than the triangle-free systems. This is expected since creating a new triangle is possible starting from a parent structure with any ring size.

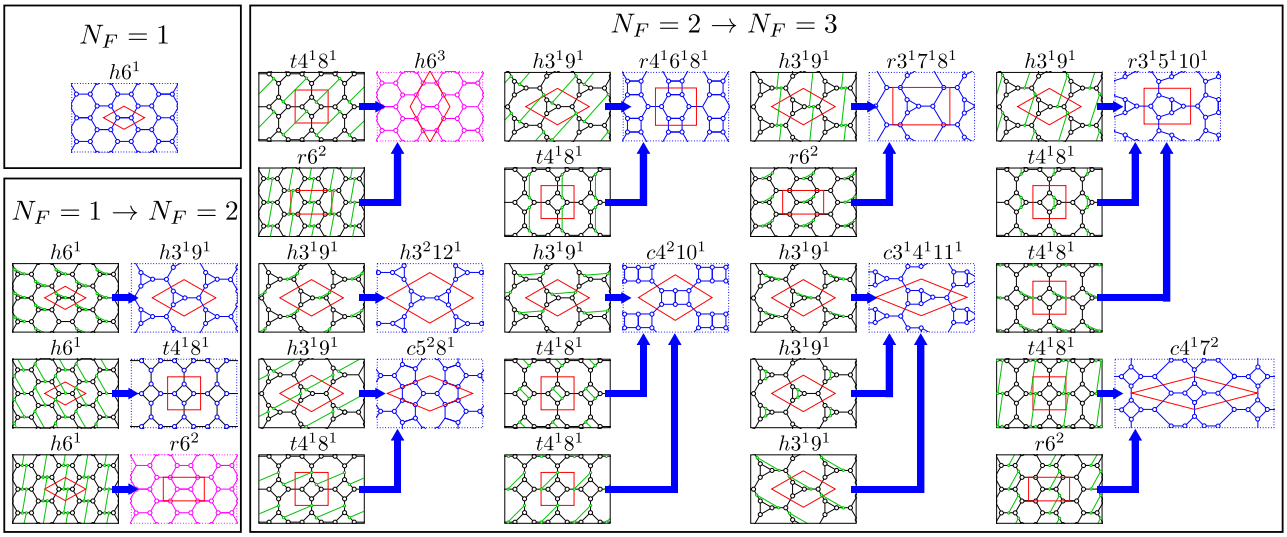


Fig. 9 | Illustrations of the add-dimer algorithm method. Starting from the parent structure $h6^1$ (the only structure with $N_F = 1$), the first generation of children includes $h3^1 9^1$, $t4^1 8^1$, and $r6^2$ (this last one being a rectangular conventional cell of the smaller $h6^1$ system, and therefore does not count as a new structure), which themselves generate 7, 6, and 3 children at generation 3 (after excluding repeated outcomes). A new generation (corresponding to the blue and purple structures) is obtained by inserting dimers at all the possible positions in the parent structure modulo symmetry, as shown by green lines. While unique primitive cells are only included in the final dataset, the non-primitive systems (purple structures), like $r6^2$ and $h6^3$, are required as parents for future generations. Even for three generations, multiple duplicates are generated and are dismissed because their embedding tensors are isomorphic. While ball-and-stick models are shown here for illustration purposes, this search is completely coordinate-free, as a dimer can be added directly to a parent tensor to create each child. CIF files are available in the online repository for a selected number of structures[33] and the generation code is provided in ref. 41.

Table 4 | Number of structures obtained using the embedding tensor-based add-dimer algorithm per type of symmetry group and by number of faces, considering systems with no triangles

Symmetry Group	Number of faces							Total
	1	2	3	4	5	6	7	
p1	0 (0)	0 (0)	0 (0)	1 (7)	13 (95)	115 (953)	857 (8720)	986 (9775)
cm	0 (0)	0 (0)	0 (1)	2 (12)	7 (37)	49 (210)	170 (833)	228 (1093)
pm	0 (0)	0 (0)	0 (2)	0 (3)	6 (35)	23 (128)	124 (653)	153 (821)
p2	0 (0)	0 (0)	0 (0)	2 (2)	9 (12)	35 (57)	136 (252)	182 (323)
cmm	0 (0)	0 (0)	3 (3)	4 (4)	20 (30)	25 (32)	89 (155)	141 (224)
pmm	0 (0)	0 (0)	1 (1)	4 (6)	11 (12)	24 (42)	64 (89)	104 (150)
pmg	0 (0)	0 (0)	0 (0)	1 (2)	0 (0)	10 (17)	0 (0)	11 (19)
p3m1	0 (0)	0 (1)	0 (0)	1 (1)	1 (5)	0 (3)	4 (7)	6 (17)
p31m	0 (0)	0 (0)	0 (0)	0 (1)	0 (1)	2 (2)	1 (5)	3 (9)
p6m	1 (1)	0 (0)	0 (1)	0 (0)	0 (0)	1 (2)	2 (2)	4 (6)
p4m	0 (0)	1 (1)	0 (0)	0 (0)	0 (0)	3 (4)	0 (0)	4 (5)
pg	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (5)	0 (0)	0 (5)
p3	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (2)	0 (2)
p4g	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	1 (1)	0 (0)	1 (1)
Total	1 (1)	1 (2)	4 (8)	15 (38)	67 (227)	288 (1456)	1447 (10718)	1823 (12450)

The total number of structures including triangles are shown in parenthesis.

In summary, the use of the embedding tensor not only provides a coordinate-free framework but also fully exploits the symmetries of parent structures to guide the efficient iterative discovery of new ones.

Coordinate generation

Although we have spent most of this paper iterating what is possible without coordinates, retrieving aspects such as finding the energy of the system or plotting the structure requires generating coordinates for the vertices and drawing the connections of the edges. However, since connectivity is insufficient to describe these structural details, we must store not only the atomic connections, but whether they cross the borders of the unit cell. That information is readily available in the embedding tensor. The connections consist of two matrices for the $x-$ and $y-$ axes: $X_{ij} = 1$ if the connection from $\vec{x}_i \rightarrow \vec{x}_j$ crosses the unit cell in the positive x direction, for atoms i and j. If it crosses in the negative x direction, $X_{ij} = -1$ instead. This also means the matrices are antisymmetric and sparse. In order to keep track of whether an edge crosses the borders of the unit cell as dimers are added, the coordinates and connections for the initial hexagonal system ($h6^1$) are predefined. Then, when a dimer is added that cuts a face, the resulting edges that are cut by this action are recalculated and added back in. This process

can continue as more dimers are added, and this also leads to a set of starting coordinates.

Normally, generating the relaxed system requires successive iterations, taking a candidate solution with the required connectivity and moving every atom in the direction of lowest energy, or the negative of the gradient of the energy. It is possible to recast the problem to avoid an iterative procedure, and obtain lattice vectors as well. For this, we use a simple potential in fractional coordinates that makes every atom tend to be the center of its nearest-neighbors, while respecting the loop conditions. If we take the (initially unknown) coordinates for each atom $\vec{x}_i$ and the adjacency matrix A_{ij}, the average of the position of the neighbors of atom (ignoring edges that cross the unit cell) i is $\frac{1}{deg(v_i)}\sum_j A_{ij}\vec{x}_j$. To include the contributions from the edges that cross the unit cell, we use the X and Y matrices defined earlier. Since the image of point x_j from point x_i is $(x\prime = x_j + X_{ij})$, the result is $\frac{1}{deg(v_i)}\sum_j (A_{ij}\vec{x}_j + X_{ij})$. Next, we calculate the distance an atom is from the average of its neighbors, $\frac{1}{deg(v_i)}\sum_j (A_{ij}\vec{x}_j + X_{ij}) - \vec{x}_i$. We define a simple energy function to be

$$U = ||L_{ij}\vec{x}_j - \vec{b}_i||^2 \tag{16}$$

where $b_i = \sum_j (X_{ij})$, and $L_{ij} = A_{ij} - \deg(v_i)\delta_{ij}$ is the graph Laplacian. In this form, we can solve for x_i as a least-squares problem, yielding a solution without iterations. While not exact, this approximation is excellent in practice to get a set of starting coordinates. It has the advantage of not requiring absolute distance as the potential can be calculated using fractional coordinates with no a priori knowledge of the lattice vectors.

The question to answer is: What does a *good* unit cell look like? When we apply a matrix with determinant 1, edges tend to get compressed in one direction and expanded in another. So in an ideal case, edges would have roughly equal orientations and lengths. So let's consider an ideal case where a vertex is located at the origin connected to other vertices equally spread around the unit circle. To show the length and orientations of a vector, we use its tensor product $vv^\top$. If we have a vector at a specific angle, then

$$v(\theta) = \begin{bmatrix} cos(\theta) \\ sin(\theta) \end{bmatrix} \tag{17}$$

It follows that

$$v(\theta)v(\theta)^\top = \begin{bmatrix} \cos(\theta)^2 & \cos(\theta)\sin(\theta) \\ cos(\theta)sin(\theta) & sin(\theta)^2 \end{bmatrix} \tag{18}$$

When we take the limit as the number of connections goes to infinity, which approximates lines of all possible angles represented equally, then we can integrate over θ to get the net result. Integrating over θ from $0 \rightarrow \tau$ yields

$$\int_0^\tau v(\theta)v(\theta)^\top d\theta = \frac{1}{2}\tau\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{19}$$

This represents the ideal case as, in all likelihood, we get a warped version of this, so let's see how the result changes under these new conditions. Here, the vector $\vec{v}$ will be acted upon by an unknown matrix M to form $M\vec{v}$. This will result in an operation looking like this instead:

$$\int_0^\tau Mv(\theta)v(\theta)^\top M^\top d\theta = \frac{1}{2}\tau I \tag{20}$$

And now we can take the matrices out of the integral. Additionally, since a rotation does not change how good the unit cell is, we can make the matrix symmetric: $M^\top = M$. The operation therefore becomes:

$$M \cdot \int_0^\tau v(\theta)v(\theta)^\top d\theta \cdot M = \frac{1}{2}\tau I. \tag{21}$$

We can multiply by M^{-1} on the left and right, resulting in

$$\int_0^\tau v(\theta)v(\theta)^\top d\theta = \frac{1}{2}\tau M^{-2}. \tag{22}$$

and finally

$$\left(\frac{2}{\tau}\int_0^\tau v(\theta)v(\theta)^\top d\theta\right)^{-\frac{1}{2}} = M. \tag{23}$$

However, in a real unit cell, we sum over the vectors in our graph G, leading to

$$\left(\frac{2}{\tau}\sum_{ij\in G} v_{ij}v_{ij}^\top\right)^{-\frac{1}{2}} = M. \tag{24}$$

In practice one still has to account for the wrapping around the unit cell, using the X and Y matrices discussed above. We also note that this method is agnostic to any affine transformation, which means solving for the fractional coordinates and solving for the unit cells are separable. For this reason, one can rescale the unit cell vector and positions so that the average bond length is, *e.g.*, that of graphene.

So we can take vectors $\vec{v}$, multiply by this matrix, and it will result in a good approximation of the underlying unit cell. Due to the symmetry of our initial setup, if a surface has this symmetry, the matrix will respect it, and the outputs will be identical. In cases without any symmetry, however, (*i.e.*, $p1$), it is slightly less accurate, not accounting for volume or scaling factors.

Note that the above is only an approximation to the true interatomic forces, and more complex interactions are required to obtain high-fidelity results. But as an approximation or for visualization purposes, these methods work remarkably well at a very modest computational cost. Note also, that aside from imposing a threefold connectivity constraint, no other chemically-inspired constraints are applied. This means that some of the tilings, while valid from a topological perspective, may not yield chemically stable structures. The final determination is quite straightforward to make using the coordinates and the connections determined in the search, for example using more realistic potentials such as AIREBO[34] or GAP[35].

Discussion

We have introduced the concept of embedding tensor, a powerful mathematical object that describes tilings of a surface by polygons. By elevating the status of faces and edges to that of vertices, we developed means to uniquely describe structures, perform exhaustive searches, and find their wallpaper symmetry. Because operations involve only integer arithmetic, the search is precise and numerically fast. Once the search is performed, it is also possible to obtain good approximations of atomic coordinates for further studies, such as electronic and vibrational properties. In general, the embedding tensor provides an ideal framework for high-throughput computational studies of 2D structures, beyond carbon-based nanostructures.

As shown in "Regular Polytopes" by Coxeter[22], relaxing the requirement of volume for polyhedra creates a wealth of shapes, all with vertex to vertex, edge to edge, and face to face mappings, more than the original five platonic solids. This includes the Petrie-Coxeter polyhedra (page 24), star-polyhedra (page 93), and flat tilings (page 59), among others. With the large number of symmetries involved in polyhedra, it is likely that the information required to build one could be described with the tensor, and they could be explicitly enumerated.

Methods

Generation and analysis of the embedding tensor

The main goal of this paper is to design a fundamental framework to handle tasks including (but not limited to) symmetry identification, isomorphism detection, and structure generation. While this is a methodological development per se, the basic tools to the practical implementation of the new

framework consist of elementary geometry[22,36], graph theory concepts[37], basic linear algebra operations[38] and a programming language to implement it in: here the implementations use FORTRAN[39] and PYTHON[40]. The calculations are mainly done with integers and are not particularly affected by precision issues related to float point algebra.

Data availability

All the embedding tensors of possible tilings up to NF = 7 are provided for free use in the online Mendeley dataset repository, along with 97 CIF files of all the systems that have pentagonal (and higher) rings (https://doi.org/10.17632/8ng8tb49yn.1). The code used to generate the tensors during the structural search is available at https://github.com/acm3851/embedding_tensor.

Code availability

The code used to generate the tensors during the structural search is available in ref. 41.

Received: 22 July 2025; Accepted: 12 December 2025;
Published online: 14 January 2026

References

1. Meunier, V. et al. Carbon science perspective in 2024: current research and future challenges. *Carbon* **229**, 119488 (2024).
2. Das Sarma, S., Adam, S., Hwang, E. H. & Rossi, E. Electronic transport in two-dimensional graphene. *Rev. Mod. Phys.* **83**, 407–470 (2011).
3. Cohen, M. L. & Louie, S. G. *Fundamentals of Condensed Matter Physics* (Cambridge University Press, 2016).
4. Crespi, V. H., Benedict, L. X., Cohen, M. L. & Louie, S. G. Prediction of a pure-carbon planar covalent metal. *Phys. Rev. B* **53**, R13303–R13305 (1996).
5. Terrones, H. et al. New metallic allotropes of planar and tubular carbon. *Phys. Rev. Lett.* **84**, 1716–1719 (2000).
6. Lusk, M. T. & Carr, L. Creation of graphene allotropes using patterned defects. *Carbon* **47**, 2226–2232 (2009).
7. Chen, M., Han, X. & Tang, K. Topological regulations of stone-wales graphene. *Carbon* **226**, 119163 (2024).
8. Rao Nulakani, N. V., Kamaraj, M. & Subramanian, V. Coro-graphene and circumcoro-graphyne: novel two-dimensional materials with exciting electronic properties. *RSC Adv.* **5**, 78910–78916 (2015).
9. Li, X., Wang, Q. & Jena, P. ψ-graphene: a new metallic allotrope of planar carbon with potential applications as anode materials for lithium-ion batteries. *J. Phys. Chem. Lett.* **8**, 3234–3241 (2017).
10. Paz, M. L. A., Saraiva-Souza, A., Meunier, V. & ao, E. C. G. Naphthylenes: 1D and 2D carbon allotropes based on naphthyl units. *Carbon* **153**, 792 – 803 (2019).
11. Wang, X. et al. Qpht-graphene: a new two-dimensional metallic carbon allotrope. *Phys. Lett. A* **381**, 2845–2849 (2017).
12. Hosseini, M., Soleimani, M., Shojaei, F. & Pourfath, M. Graphsene as a novel porous two-dimensional carbon material for enhanced oxygen reduction electrocatalysis. *Sci. Rep.* **14**, 9129 (2024).
13. Bhattacharya, D. & Jana, D. Tpdh-graphene: a new two dimensional metallic carbon with NDR behaviour of its one dimensional derivatives. *Phys. E Low Dimens. Syst. Nanostruct.* **127**, 114569 (2021).
14. Girão, E. C., Macmillan, A. & Meunier, V. Classification of sp2-bonded carbon allotropes in two dimensions. *Carbon* **203**, 611–619 (2023).
15. Wang, Z. et al. Phagraphene: a low-energy graphene allotrope composed of 5-6-7 carbon rings with distorted dirac cones. *Nano Lett.* **15**, 6182–6186 (2015).
16. Fan, X., Li, J. & Chen, G. New carbon allotropes with metallic conducting properties: a first-principles prediction. *RSC Adv.* **7**, 17417–17426 (2017).
17. Lyakhov, A. O., Oganov, A. R., Stokes, H. T. & Zhu, Q. New developments in evolutionary structure prediction algorithm uspex. *Comput. Phys. Commun.* **184**, 1172–1182 (2013).
18. Wang, Y., Lv, J., Zhu, L. & Ma, Y. Calypso: a method for crystal structure prediction. *Comput. Phys. Commun.* **183**, 2063–2070 (2012).
19. Wang, H. et al. Calypso structure prediction method and its wide application. *Comput. Mater. Sci.* **112**, 406–415 (2016).
20. Yi, W.-C., Liu, W., Botana, J., Liu, J.-Y. & Miao, M.-S. Microporosity as a new property control factor in graphene-like 2d allotropes. *J. Mater. Chem. A* **6**, 10348–10353 (2018).
21. Bullard, Z., Girao, E. C., Daniels, C., Sumpter, B. G. & Meunier, V. Quantifying energetics of topological frustration in carbon nanostructures. *Phys. Rev. B* **89**, 245425 (2014).
22. Coxeter, H. *Regular Polytopes*. Dover Books on Advanced Mathematics (Dover Publications, 1973).
23. Hudspeth, M. A., Whitman, B. W., Barone, V. & Peralta, J. E. Electronic properties of the biphenylene sheet and its one-dimensional derivatives. *ACS Nano* **4**, 4565–4570 (2010).
24. Fan, Q. et al. Biphenylene network: a nonbenzenoid carbon allotrope. *Science* **372**, 852–856 (2021).
25. Long, G. et al. Theoretical investigation on two-dimensional non-traditional carbon materials employing three-membered ring and four-membered ring as building blocks. *Carbon* **95**, 1033–1038 (2015).
26. Wang, S., Yang, B., Chen, H. & Ruckenstein, E. Popgraphene: a new 2D planar carbon allotrope composed of 5-8-5 carbon rings for high-performance lithium-ion battery anodes from bottom-up programming. *J. Mater. Chem. A* **6**, 6815–6821 (2018).
27. Hayat, S., Imran, M. & Liu, J.-B. Correlation between the estrada index and π-electronic energies for benzenoid hydrocarbons with applications to boron nanotubes. *Int. J. Quantum Chem.* **119**, 26016 (2019).
28. Gallier, J. & Xu, D. A guide to the classification theorem for compact surfaces (2012).
29. Song, Q. et al. Graphenylene, a unique two-dimensional carbon network with nondelocalized cyclohexatriene units. *J. Mater. Chem. C.* **1**, 38–41 (2013).
30. Farris, F. *Creating Symmetry: The Artful Mathematics of Wallpaper* (Princeton University Press, 2015).
31. Togo, A. & Tanaka, I. Spglib: a software library for crystal symmetry search 1808.01590. (2018).
32. Silva, P. V., Fadel, M., Souza Filho, A. G., Meunier, V. & Girão, E. C. Tripentaphenes: two-dimensional acepentalene-based nanocarbon allotropes. *Phys. Chem. Chem. Phys.* **22**, 23195–23206 (2020).
33. Macmillan, L., Girao, E. C. & Meunier, V. Embedding tensor for two-dimensional carbon allotropes. https://doi.org/10.17632/8ng8tb49yn.1 (2025).
34. Stuart, S. J., Tutein, A. B. & Harrison, J. A. A reactive potential for hydrocarbons with intermolecular interactions. *J. Chem. Phys.* **112**, 6472–6486 (2000).
35. Deringer, V. L. & Csányi, G. Machine learning based interatomic potential for amorphous carbon. *Phys. Rev. B* **95**, 094203 (2017).
36. Farris, F. *Creating Symmetry: The Artful Mathematics of Wallpaper Patterns* (Princeton University Press, 2015).
37. Diestel, R. *Graph Theory* 5th edn (Springer, 2017).
38. Strang, G. *Introduction to Linear Algebra* 4th edn (Wellesley-Cambridge Press, 2009).
39. Metcalf, M., Reid, J., Cohen, M. & Bader, R. *Modern Fortran Explained: Incorporating Fortran 2023*. Numerical Mathematics and Scientific Computation (Oxford University Press, 2023).
40. Matthes, E. *Python Crash Course: A Hands-On, Project-Based Introduction to Programming* 2nd edn (No Starch Press, 2019).
41. Mcmillan, L. Embedding tensor (beta) https://github.com/acm3851/embedding_tensor (2025).

Author contributions

V.M. conceived the research. All three authors (L.M., E.C.G., and V.M.) carried out the simulations and data analysis. L.M. developed the theoretical model. All three authors interpreted the results and wrote the manuscript.

Competing interests
The authors declare no competing interests.

Additional information
Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41524-025-01932-8.

Correspondence and requests for materials should be addressed to Vincent Meunier.

Reprints and permissions information is available at http://www.nature.com/reprints

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026